

BEGINNER-FRIENDLY PDF EDITION

Configuration & Credentials

How The Ontology Machine connects local software to a real corpus, a real model provider, and a real workflow root.

Core rule: identify the layer first; fix only that layer. Configuration problems become expensive when unrelated state is deleted.

Prepared from 09_Configuration_Credentials.md. Revised for readability, clean tables, rendered ownership map, and operator-ready troubleshooting.

Contents

9.1	Source Of Truth	4
9.2	The Two Frontend Servers.....	5
9.3	Client Frontend App Home.....	6
9.4	Client Frontend Config File.....	8
9.5	Config Page Sections	9
9.6	SQL Database Path	12
9.7	Pipeline Root Folder	13
9.8	Artifact Tree Context.....	14
9.9	Client Frontend Credentials	15
9.10	OAuth	16
9.11	Provider Catalog	17
9.12	Embeddings.....	18
9.13	Admin Password	19
9.14	Orchestrator Runtime Settings	20
9.15	Orchestrator Credentials	21
9.16	Kernel Credential Model.....	22
9.17	MCP Server Configuration.....	23
9.18	Runtime Environment Variables.....	24
9.19	Typical Setup Failures	26
9.20	Safe Reset Rules	29
9.21	Configuration Checklist	32
9.22	What Configuration Does Not Do.....	34
9.23	Operator Rule	35

Recommended first pass for new operators: read the mental model, Source of Truth, the two frontend servers, credential boundaries, then use the failure or reset section that matches the symptom.

9. Configuration & Credentials

Configuration is where The Ontology Machine connects the local software to a real corpus, a real model provider and a real workflow root.

This chapter matters because there is not one single config file that controls everything. The visible user setup is mostly in the Client Frontend. Direct pipeline execution also has Orchestrator-owned runtime settings and credentials. The Kernel does not own provider secrets. The MCP Server does not own provider secrets. Corpus DBs and Artifact Trees are selected by path and workflow state, not by a remote account.

The shortest safe mental model is:



Figure 1. Configuration ownership map: each box is a separate state and credential boundary.

Do not flatten those layers in your head. A Query Agent can be ready while a direct Orchestrator OCR run is not ready. A DB can be queryable while embeddings are unavailable. A Taxonomy Agent chat can be live while a Kernel workflow is blocked on a missing Pipeline Root Folder.

9.1. Source Of Truth

The active configuration implementation is spread across these owner surfaces:

Surface	Main Files
Client Frontend config defaults	Client Frontend/client_frontend/config/types.js
Client Frontend config workflow	Client Frontend/client_frontend/config/
Client Frontend config UI	Client Frontend/src/config.html, Client Frontend/client_frontend/browser/config_app/
Client Frontend app home	Client Frontend/client_frontend/app_paths/
Client Frontend credentials	Client Frontend/client_frontend/credentials/
Client Frontend secret vault	Client Frontend/client_frontend/vault/
Client Frontend provider catalog	Client Frontend/shared/provider-catalog.json
Client Frontend HTTP config routes	Client Frontend/client_frontend/http/config_workflow.js
Client Frontend OAuth routes	Client Frontend/client_frontend/http/credentials_workflow.js
Orchestrator runtime settings	00 - Orchestrator/orchestrator/state/repository.py
Orchestrator credential subsystem	00 - Orchestrator/orchestrator/credentials/
Orchestrator credentials UI	00 - Orchestrator/orchestrator/ui/surface_actions_credentials.py
Orchestrator model settings UI	00 - Orchestrator/orchestrator/ui/model_settings_layout.py
Kernel LLM bridge	00 - Orchestrator/orchestrator/orchestrator_contract/kernel_llm.py

If behavior in the GUI and this chapter disagree, the files above win.

9.2. The Two Frontend Servers

The Client Frontend has two server modes.

Server	URL	Source Launcher	Purpose
Chat server	<code>http://127.0.0.1:3000</code> by default	<code>Client Frontend\start.bat</code>	Query Agent, Ontology Agent, Taxonomy Agent, source viewer
Config server	<code>http://127.0.0.1:3001/config</code>	<code>Client Frontend\config.bat</code>	DB path, providers, credentials, prompts, policy

They are different processes. The config server being open does not mean the chat server is running. If the chat UI reports `Failed to fetch`, first check whether the chat server is actually running. A very common operator mistake is to start only `config.bat`, see port `3001`, and assume the agent server on port `3000` is alive.

The startup script writes logs under the Client Frontend app home:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\logs\  
startup.log  
startup-browser-helper.log  
config-startup.log  
config-browser-helper.log
```

It also writes separate server-state files:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\state\  
server-chat.json  
server-config.json
```

Those state files are used during stale server cleanup. If a server fails to open a browser or a port cannot be released, inspect the matching startup log and server-state file before changing configuration.

9.3. Client Frontend App Home

The Client Frontend stores user-local state in an app home. Resolution order:

1. explicit `appHome` passed by code or tests
2. `VISION_PIPELINE_CLIENT_FRONTEND_HOME`
3. `%LOCALAPPDATA%\Enterprise Stack\Client Frontend`

Normal installed or source usage uses:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend
```

The layout is:

```
Client Frontend app home\  
  config\  
    config.json  
    frontend_policy.json  
    .salt  
  state\  
    chats.db  
    chats.db-wal  
    chats.db-shm  
    credentials_state.json  
    keystore.enc  
    keystore.lock  
    oauth_token.enc  
    oauth_token.lock  
    oauth_latest_report.json  
    server-chat.json  
    server-config.json  
    ontology_agent\  
    ontology_agent_kernel\  
    pipeline_manager\  
  logs\  
    startup.log  
    startup-browser-helper.log  
    config-startup.log  
    config-browser-helper.log
```

The source module also has a `state-snapshot` folder used by deployment and installer support flows, but day-to-day user configuration lives in app home.

Important consequences:

- deleting the install folder does not necessarily delete user config
- reinstalling can reuse the same app home
- copying `keystore.enc` without the matching `config\.salt` can make frontend secrets unreadable
- deleting `chats.db` removes local chat history, not the Corpus DB

- deleting `server-chat.json` or `server-config.json` is only safe after the corresponding server process is closed
- `*.db-wal` and `*.db-shm` sidecars should not be deleted while the chat store is open

9.4. Client Frontend Config File

The visible frontend config is stored in:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\config\config.json
```

Default values come from `Client Frontend/client_frontend/config/types.js`.

Current defaults:

Field	Default	Meaning
<code>customer_name</code>	<code>Vision Pipeline Case Worker</code>	Display/persona label used by the frontend
<code>sql_database_path</code>	<code>..\SampleDB\Consciousness Travel - Default Demo\Corpus\corpus.db</code>	Active Corpus DB for Query and Ontology work
<code>pipeline_root</code>	empty, inferred when possible	The Ontology Machine root used for Kernel/MCP workflows
<code>llm_provider</code>	<code>openai</code>	Frontend agent LLM provider
<code>llm_base_url</code>	<code>https://api.openai.com/v1</code>	Frontend agent LLM base URL
<code>llm_model</code>	<code>gpt-5.4</code>	Frontend agent LLM model
<code>embedding_provider</code>	<code>openai</code>	Frontend embedding provider
<code>embedding_base_url</code>	<code>https://api.openai.com/v1</code>	Frontend embedding base URL
<code>embedding_model</code>	<code>text-embedding-3-small</code>	Frontend embedding model
<code>port</code>	<code>3000</code>	Chat server port
<code>theme</code>	<code>dark</code>	Main frontend theme
<code>admin_secret</code>	empty	Optional config-page protection
<code>context_limit</code>	<code>127096</code>	Frontend agent context budget

Secrets may appear as masked or encrypted values in stored config, but the current runtime path migrates API keys into the frontend keystore under `state\keystore.enc` and then keeps saved `llm_api_key` and `embedding_api_key` empty in `config.json`.

That is normal. A config file without visible API keys does not mean the keys are missing.

9.5. Config Page Sections

The config page is divided into four user-facing sections:

Setup

Paths, port, shell and active DB.

Models

Providers, model names, API keys and OAuth status.

Prompts

Prompt families for future agent construction.

Advanced Policy

Limits, catalog behavior and low-level runtime tuning.

Setup

Setup contains the local path and shell settings:

Field	Use
Customer name	UI/persona label
SQL database path	active Corpus DB for Query Agent and Ontology Agent
Pipeline Root Folder	root folder containing the Ontology Machine modules
Port	chat server port
Theme	light/dark UI theme

The SQL database path is validated on save. The frontend opens the selected DB read-only and requires at least the `documents` table. If the file cannot be opened or is not a Corpus DB, save fails with `field: sql_database_path`.

Models

Models contains:

- OAuth status
- LLM provider
- LLM base URL

- LLM API key
- LLM model
- context limit
- embedding provider
- embedding base URL
- embedding API key
- embedding model
- admin password

The LLM block controls the three frontend agents:

- Query Agent
- Ontology Agent
- Taxonomy Agent chat surface

The embedding block controls embedding-dependent frontend and ontology operations. Existing corpora can still be inspected without embedding credentials. Missing embeddings are a degraded retrieval state, not DB corruption.

Prompts

Prompts edits the agent prompt sections stored inside:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\config\frontend_policy.json
```

This is where the Query Agent and Ontology Agent prompt sections are edited. The prompt UI edits one prompt family at a time so the config page does not turn into one giant wall of text.

Prompt edits affect future agent construction. If the chat server is already running, saving config reloads the active frontend agents.

Advanced Policy

Advanced Policy also writes to:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\config\frontend_policy.json
```

It controls lower-level frontend behavior:

- chat history limits

- memory summarization limits
- model catalog source order
- Query Agent runtime limits
- max tool rounds
- max SQL rows
- max text lengths
- workbench output limits
- prompt section defaults

This is a power-user surface. It is useful for hardening or tuning, but most normal users should not need to touch it.

9.6. SQL Database Path

`sql_database_path` is the active DB for the Client Frontend agents.

It is used by:

- Query Agent reads
- Ontology Agent reads and ontology-layer writes
- source viewer
- page image viewer
- DB health badges
- Base Graph status badge
- ontology lens count badge

It is not merely a display path. If it points at the wrong DB, the agents will answer against the wrong corpus.

Relative paths are resolved from the Client Frontend module root. In the normal source or installed tree:

```
Client Frontend\  
  config.bat  
  start.bat  
SampleDB\  
  Consciousness Travel - Default Demo\  
    Corpus\  
      corpus.db
```

the default relative path works:

```
..\SampleDB\Consciousness Travel - Default Demo\Corpus\corpus.db
```

For production or handover, absolute paths are safer:

```
C:\Users\...\The Ontology Machine\SampleDB\Consciousness Travel - Default  
Demo\Corpus\corpus.db  
F:\Sample DB\Research Corpus\Artifact Tree\Corpus\corpus.db
```

The selected DB can be any compatible Corpus DB. It does not have to be the DB created by the most recent workflow. The user is responsible for selecting the intended corpus.

9.7. Pipeline Root Folder

`pipeline_root` points to the Ontology Machine root. It should be the folder that contains the module folders:

```
00 - Orchestrator
01 - Optimizer
02 - Interpreter
03 - Validator
04 - Normalizer
05 - Corpus Builder
06 - Edit Suite
07 - MCP Server
08 - Semantic Control Kernel
Client Frontend
```

The frontend can infer this root when `Client Frontend` is installed as a child of the root and sibling folders such as `07 - MCP Server` and `08 - Semantic Control Kernel` exist. If it cannot infer the root, set `Pipeline Root Folder` manually.

This path is needed by:

- Taxonomy Agent / Kernel workflows
- local MCP bridge startup
- Ontology Agent Kernel calls such as deterministic Base Graph mining
- workflow status inspection
- Kernel event polling

If the Pipeline Root Folder is missing or wrong, the Query Agent may still work because it only needs the active DB. The Taxonomy Agent will fail or report:

```
Choose Pipeline Root Folder
```

or an MCP/Kernel startup error.

9.8. Artifact Tree Context

The config page does not directly select an Artifact Tree. It selects:

- a Corpus DB through `sql_database_path`
- the Ontology Machine root through `pipeline_root`

The Artifact Tree is inferred or collected by workflows.

For Query Agent and Ontology Agent work, the active DB is enough as long as the DB is compatible. For Kernel-governed ingest, reset, rebuild and merge, the DB must be part of a valid Artifact Tree target. Kernel workflows expect the DB to live under:

```
Artifact Tree\Corpus\<database>.db
```

and use Kernel-owned dialogs or binding state to prove the surrounding Artifact Tree.

This distinction is important:

- readable DB path: enough for Query/Ontology inspection
- governed Kernel target: DB plus valid Artifact Tree identity

A DB outside an Artifact Tree can be queried. It should not be treated as a valid ingest/reset/rebuild/merge target.

9.9. Client Frontend Credentials

The frontend has two credential targets:

Target	Stored As	Used For
llm_shared	LLM API key or OpenAI OAuth session	Query Agent, Ontology Agent, Taxonomy Agent chat
embeddings	Embedding API key	semantic/vector retrieval helpers and embedding generation paths exposed through frontend tools

Credential status is exposed to the config page as:

```
credential_state.targets.llm_shared
credential_state.targets.embeddings
credential_state.oauth_session
```

Secrets are stored in:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\state\keystore.enc
```

The portable frontend vault uses:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\config\.salt
```

Do not separate `keystore.enc` from `.salt` when copying frontend app-home state. Without the matching salt, encrypted frontend secrets cannot be read.

The non-secret status file is:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\state\credentials_state.json
```

That file says whether a secret exists. It is not the secret itself.

9.10. OAuth

Frontend OAuth is supported for the OpenAI LLM path.

The config page routes are:

```
GET /config/oauth/login
GET /config/oauth/callback
POST /config/api/oauth/logout
```

When OAuth is connected and the selected LLM provider supports it, the LLM path primarily uses OAuth. The stored API key remains a fallback if one exists.

Important details:

- OAuth is an LLM path, not an embeddings path.
- OAuth support depends on the selected provider.
- If the selected provider does not support the frontend OAuth path, the login button is disabled or the route returns an unsupported-provider message.
- The LLM test endpoint is disabled while an OAuth session is active.
- OAuth token state is local user state.

Frontend OAuth state lives under:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\state\  
  oauth_token.enc  
  oauth_token.lock  
  oauth_latest_report.json
```

Frontend OAuth is separate from Orchestrator OAuth. Logging into one does not configure the other.

9.11. Provider Catalog

The frontend provider catalog lives in:

```
Client Frontend\shared\provider-catalog.json
```

It defines:

- provider IDs
- display names
- API families
- default base URLs
- LLM support
- embedding support
- OAuth support
- whether an API key is optional
- fallback model lists
- context limits
- pricing hints

Current provider IDs include:

openai anthropic google xai openrouter groq together fireworks mistral
deepseek sambanova cerebras mammoth lmstudio ollama openai_compat

Not every provider supports every role. For example, some providers support LLM calls but not embeddings. Local providers such as LM Studio or Ollama may allow optional API keys. The UI reflects that through readiness messages.

Model lists can come from:

- saved catalog state
- provider model catalog refresh
- fallback seed models from the provider catalog
- manually entered/saved model values

If model refresh fails, a configured fallback model can still be valid if the provider accepts it. A failed catalog refresh is not automatically a failed provider connection.

9.12. Embeddings

Embeddings are optional for basic DB inspection but important for semantic search, retrieval quality and ontology embedding refresh.

There are three different states that must not be confused:

State	Meaning
no embedding credentials	the system cannot generate new embeddings through that provider
no embeddings in DB	the corpus has no vector materialization yet
embeddings unavailable warning	retrieval can fall back to SQL/FTS/lexical paths, but semantic/vector behavior is degraded

Missing embeddings are not DB corruption.

A corpus can be valid with no embeddings. The Query Agent should still use SQL, FTS, documents and ontology tables where available. Semantic search may fall back or warn depending on the active DB and provider state.

Embedding credentials are needed when the user wants to:

- regenerate corpus embeddings
- generate embeddings for newly created ontology objects
- improve semantic/vector retrieval over a corpus that lacks vectors

If no embeddings API key is set, the user-facing message should explain that semantic/vector search and embedding generation are degraded, not that the DB is broken.

9.13. Admin Password

`admin_secret` is optional. If it is empty, config changes are not locked. If it is set, the config page requires an admin unlock before saving, testing connections or refreshing model catalogs.

Admin unlock creates a short-lived local admin session cookie. The current implementation refreshes that session while it is used.

Admin protection is local UI protection. It is not a network security layer for running The Machine on an untrusted host. The servers bind to loopback `127.0.0.1`, and the product assumes a local user/operator environment.

If the admin password is forgotten, the safe recovery path is:

1. close the config and chat servers
2. back up the Client Frontend app home
3. remove or regenerate the frontend config state
4. start the config server again and save a new admin password

Do not delete Artifact Trees or Corpus DBs to reset the admin password. They are not involved.

9.14. Orchestrator Runtime Settings

Direct Orchestrator runs use Orchestrator-owned runtime settings:

```
00 - Orchestrator\state\runtime_settings.json
```

The Orchestrator settings are separate from Client Frontend `config.json`.

The runtime settings contain:

Section	Default	Used For
<code>llm_shared_provider</code>	OpenAI, <code>https://api.openai.com/v1</code>	Interpreter and Normalizer provider
<code>embeddings_provider</code>	OpenAI, <code>https://api.openai.com/v1</code>	Corpus Builder embeddings provider
<code>optimizer_ocr_provider</code>	OpenAI, <code>https://api.openai.com/v1</code>	Optimizer OCR provider
<code>interpreter</code>	<code>gpt-5.4</code> , <code>max_output_tokens=8000</code>	extraction/interpreter model calls
<code>normalizer</code>	<code>gpt-5.4-mini</code> , <code>max_output_tokens=15000</code>	normalization model calls
<code>corpus_builder_embeddings</code>	<code>text-embedding-3-small</code>	embedding generation
<code>optimizer_ocr</code>	<code>gpt-5.4</code> , <code>max_output_tokens=15000</code> , <code>timeout_seconds=120</code>	OCR/vision extraction support

The Orchestrator UI writes these settings directly under `state`. The config page in the Client Frontend does not replace this surface.

9.15. Orchestrator Credentials

The Orchestrator has three credential targets:

Target	Label	Used By
llm_shared	LLM Shared API	Interpreter, Normalizer, Kernel LLM bridge through Orchestrator
optimizer_ocr	Optimizer OCR API	Optimizer OCR / vision paths
embeddings	Embeddings API	Corpus Builder <code>generate_embeddings</code>

Orchestrator credential state lives under:

```
00 - Orchestrator\state\  
  credentials_state.json  
  keystore.enc  
  keystore.lock  
  oauth_token.enc  
  oauth_token.lock  
  oauth_latest_report.json  
  admin_audit.jsonl
```

The Orchestrator keystore is Windows DPAPI-backed. It is tied to the local Windows user/machine security context. Copying `00 - Orchestrator\state` to another computer may not make secrets readable there.

Orchestrator OAuth is also local Orchestrator state. It is not the same as the Client Frontend OAuth session.

Credential readiness is capability-based:

- Interpreter needs `llm_shared`
- Normalizer needs `llm_shared`
- Optimizer OCR needs `optimizer_ocr`
- Corpus Builder embeddings need `embeddings`

Embeddings are warning-only for some flows: a missing embeddings API key should skip or degrade embedding generation, not invalidate the corpus.

9.16. Kernel Credential Model

The Semantic Control Kernel does not store API keys or OAuth tokens.

When the Kernel needs an LLM function, it routes through the Orchestrator contract:

```
Kernel
-> Orchestrator contract action
-> Orchestrator runtime settings and credentials
-> Interpreter provider call
-> Kernel LLM response capture
```

The relevant Orchestrator contract action reads:

```
00 - Orchestrator\state\runtime_settings.json
00 - Orchestrator\state\keystore.enc / oauth_token.enc
```

This is why Kernel workflows can fail with provider or credential diagnostics even when the Client Frontend chat agent itself is ready. The chat agent and Kernel owner calls are not the same credential boundary.

9.17. MCP Server Configuration

The MCP Server is a local bridge. It exposes governed tools to the Taxonomy Agent and host-only bridge tools to the Client Frontend.

Important MCP config files:

```
07 - MCP Server\config\agent_permissions.json
07 - MCP Server\config\semantic_control_kernel_bridge.json
```

`agent_permissions.json` controls which tools are visible to which agent role. It does not store provider keys.

`semantic_control_kernel_bridge.json` tells the MCP Server where the Kernel is and how to reach it. It does not store provider keys.

If MCP startup fails, check:

- Pipeline Root Folder
- MCP Server runtime
- Kernel module path
- host bridge token/event path
- MCP logs and Kernel state

Do not fix MCP startup by putting API keys into MCP config. That is the wrong owner boundary.

9.18. Runtime Environment Variables

Most operators should not set provider environment variables manually. The system creates runtime overlays when it calls owner modules.

User-relevant environment variable:

Variable	Meaning
VISION_PIPELINE_CLIENT_FRONTEND_HOME	Overrides the Client Frontend app-home path

Provider runtime overlays used by Orchestrator/owner calls include:

```
VISION_PROVIDER_ID
VISION_PROVIDER_FAMILY
VISION_PROVIDER_BASE_URL
VISION_PROVIDER_AUTH_MODE
VISION_PROVIDER_API_KEY
VISION_PROVIDER_OAUTH_ACCESS_TOKEN
VISION_PROVIDER_OAUTH_ACCOUNT_ID
VISION_PROVIDER_OAUTH_CLIENT_ID
VISION_PROVIDER_OAUTH_SESSION_ID
VISION_PROVIDER_OAUTH_SCOPE
VISION_PROVIDER_OAUTH_EXPIRES_AT
```

Legacy compatibility variables can also be set in overlays:

```
VISION_OPENAI_AUTH_MODE
VISION_OPENAI_API_KEY
VISION_OPENAI_OAUTH_ACCESS_TOKEN
VISION_OPENAI_OAUTH_ACCOUNT_ID
VISION_OPENAI_OAUTH_CLIENT_ID
VISION_OPENAI_OAUTH_SESSION_ID
VISION_OPENAI_OAUTH_SCOPE
VISION_OPENAI_OAUTH_EXPIRES_AT
OPENAI_API_KEY
OPENAI_API_BASE_URL
```

Optimizer OCR-specific overlays include:


```
OPTIMIZER_OCR_PROVIDER_ID
OPTIMIZER_OCR_PROVIDER_FAMILY
OPTIMIZER_OCR_BASE_URL
OPTIMIZER_OCR_AUTH_MODE
OPTIMIZER_OCR_API_KEY
OPTIMIZER_OCR_OAUTH_ACCESS_TOKEN
OPTIMIZER_OCR_OAUTH_ACCOUNT_ID
OPTIMIZER_OCR_MODEL
OPTIMIZER_OCR_MAX_OUTPUT_TOKENS
OPTIMIZER_OCR_TIMEOUT_SECONDS
```

These overlays are ephemeral execution context, not persistent user configuration. Persistent configuration belongs in the config page or Orchestrator state.

9.19. Typical Setup Failures

Chat UI Says Failed To Fetch

Most likely the chat server is not running.

Check:

- Did you start `Client Frontend\start.bat`?
- Are you only running `Client Frontend\config.bat`?
- Is the chat server on the configured port?
- Did stale port cleanup fail?
- Does `startup.log` contain a startup error?

Config Page Opens But Agents Do Not Answer

The config server is separate from the chat server. Open:

```
http://127.0.0.1:3000
```

or start:

```
Client Frontend\start.bat
```

LLM Ready Is Red

Check:

- LLM provider
- LLM base URL
- API key or OAuth state
- selected model
- provider support for OAuth
- config save status

If OAuth is active, the LLM test button may be disabled because the test route is API-key oriented. That does not automatically mean the OAuth session is bad.

Embeddings Are Unavailable

This is usually not DB corruption.

Check:

- embedding provider supports embeddings
- embedding base URL
- embedding API key
- embedding model
- whether the DB already has embedding rows
- whether the current operation actually needs new embeddings

The system can still answer through SQL, FTS, documents and ontology tables.

Wrong Corpus Answers

Check `SQL database path`.

The agent answers against the configured DB, not the DB you meant in your head. If two sample DBs look similar, this is easy to miss.

SQL Database Could Not Be Opened

The config save route validates the selected DB. Common reasons:

- path does not exist
- path points to a folder instead of a DB file
- DB file is locked or inaccessible
- file is not SQLite
- SQLite opens but table `documents` is missing
- relative path resolves from the Client Frontend module root, not from the directory where the browser is open

Taxonomy Agent Cannot Start Kernel Workflows

Check:

- Pipeline Root Folder
- `07 - MCP Server` exists under that root
- `08 - Semantic Control Kernel` exists under that root
- MCP runtime is present

- Kernel runtime is present
- frontend chat server was restarted after config changes

The Query Agent can still work while Taxonomy Agent workflows are unavailable.

Direct Orchestrator Run Fails On Credentials

Check Orchestrator credentials, not only Client Frontend config.

Direct Orchestrator runs use:

```
00 - Orchestrator\state\runtime_settings.json
00 - Orchestrator\state\keystore.enc
00 - Orchestrator\state\credentials_state.json
```

Frontend LLM keys do not automatically configure Orchestrator Interpreter, Normalizer, Optimizer OCR or Corpus Builder embeddings.

OAuth Logged In But Workflow Still Fails

Check which OAuth session is active.

- Frontend OAuth helps the frontend LLM path.
- Orchestrator OAuth helps Orchestrator LLM/OCR paths where supported.
- Embeddings are API-key based or provider-optional.

Logging into one surface does not log into the other.

Stale Server Cleanup Failed

The startup script refuses to kill an unrelated process on the same port. That is intentional.

Check:

- `server-chat.json` or `server-config.json`
- process ID
- executable path
- startup log
- whether another Node process owns the port

If the server-state file points to a foreign process, close that process manually or change the configured chat port.

9.20. Safe Reset Rules

Reset only the layer that is broken.

Reset Frontend Config Only

Use when:

- config page has bad paths
- admin password is forgotten
- provider settings are unusable
- prompts/policy were edited into a bad state

Safe sequence:

1. Close chat and config servers.
2. Back up:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend
```

3. Delete or edit:

```
config\config.json  
config\frontend_policy.json
```

4. Start `Client Frontend\config.bat`.
5. Save known-good settings.

Do not delete `state\keystore.enc` unless you intentionally want to remove saved frontend API keys.

Reset Frontend OAuth Only

Use the OAuth Logout button first. If manual cleanup is needed after closing servers, remove:

```
state\oauth_token.enc  
state\oauth_token.lock  
state\oauth_latest_report.json
```

Do not delete the Corpus DB.

Reset Frontend Chat History

After closing the chat server, remove:

```
state\chats.db  
state\chats.db-wal  
state\chats.db-shm
```

This removes local chat history only.

Reset Orchestrator Runtime Settings

Use when the direct Orchestrator model settings are invalid.

After closing the Orchestrator, back up and remove:

```
00 - Orchestrator\state\runtime_settings.json
```

On next start, defaults are regenerated.

Reset Orchestrator Credentials

Use when the Orchestrator should forget saved API keys or OAuth state.

After closing the Orchestrator, back up and remove only the credential files you intend to reset:

```
00 - Orchestrator\state\credentials_state.json  
00 - Orchestrator\state\keystore.enc  
00 - Orchestrator\state\keystore.lock  
00 - Orchestrator\state\oauth_token.enc  
00 - Orchestrator\state\oauth_token.lock  
00 - Orchestrator\state\oauth_latest_report.json
```

Do not delete Artifact Trees, `Corpus\*.db`, or pipeline outputs while trying to reset credentials.

Reset Server State

Use only when the server is closed and a stale state file blocks startup.

Frontend files:

```
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\state\server-chat.json  
%LOCALAPPDATA%\Enterprise Stack\Client Frontend\state\server-config.json
```

Deleting these while the server is running makes diagnostics worse.

9.21. Configuration Checklist

For a normal user who wants to query the bundled demo:

1. Start `Client Frontend\config.bat`.
2. Confirm SQL database path points to:

```
..\SampleDB\Consciousness Travel - Default Demo\Corpus\corpus.db
```

3. Confirm Pipeline Root Folder points to the Ontology Machine root or is inferred automatically.
4. Select an LLM provider, base URL and model.
5. Add an LLM API key or use OpenAI OAuth.
6. Add an embeddings key if semantic/vector search or embedding generation is needed.
7. Save.
8. Start `Client Frontend\start.bat`.
9. Check that LLM readiness is green.
10. Check that the mounted DB status shows the expected Base Graph and ontology lens state.

For a direct Orchestrator pipeline run:

1. Start `00 - Orchestrator\run.bat`.
2. Open the model/settings and credentials surfaces.
3. Configure `llm_shared_provider`, `optimizer_ocr_provider` and `embeddings_provider`.
4. Configure Interpreter, Normalizer, Optimizer OCR and Corpus Builder Embeddings models.
5. Store credentials for `llm_shared`, `optimizer_ocr` and `embeddings` where needed.
6. Confirm the target Artifact Tree and active Semantic Release.
7. Run the pipeline.

For a Taxonomy Agent / Kernel workflow:

1. Configure the Client Frontend LLM path so the Taxonomy Agent can talk.
2. Configure Pipeline Root Folder.

3. Make sure MCP and Kernel runtimes are available.
4. If the Kernel workflow calls provider-backed owner functions, also configure the relevant Orchestrator runtime settings and credentials.
5. Let Kernel dialogs collect paths and confirmations. Do not try to smuggle governed paths into the model prompt.

9.22. What Configuration Does Not Do

Configuration does not:

- create a corpus DB by itself
- attach a Semantic Release by itself
- repair an incompatible DB
- make embeddings appear in an old DB without an embedding generation run
- make Orchestrator credentials appear from Frontend credentials
- make Frontend OAuth appear in Orchestrator OAuth
- turn a readable DB outside an Artifact Tree into a governed Kernel target

Those actions belong to workflows.

9.23. Operator Rule

When something fails, identify the layer first:

Symptom	Inspect this layer first
wrong answer	SQL database path / active corpus / retrieval
agent cannot answer	frontend LLM credentials / chat server
semantic search weak	embeddings unavailable or missing in DB
workflow cannot start	Pipeline Root / MCP / Kernel
pipeline stage fails	Orchestrator runtime settings and credentials
DB cannot be edited	DB schema / lock / owner boundary

Then fix only that layer. Configuration problems become expensive when the operator starts deleting Artifact Trees, Corpus DBs or unrelated state to fix a single missing key or wrong path.